Qatar University
College of Engineering
Dept. of Computer Science & Eng.

Lab 8 Design Sequence Diagrams

Supporting tool - Visual Paradigm | 2 hours 30 minutes

Learning objectives

- Derive the messages of a design sequence diagram from the steps of a use case specification.
- Send each message to the responsible object, consistent with the operations and associations of the design class diagram.
- Model object creation, replies, self-calls, and alternative, optional or repeated behaviour with alt, opt and loop fragments.
- Revise the class diagram when a sequence diagram shows a missing operation or link.

Course learning outcomes: CLO 2 - Elicit, analyze, prioritize and document requirements for a software system. CLO 6 - Design, implement, test and present a software system in a group.

Design Sequence Diagram

A design sequence diagram shows how objects work together, in time order, to carry out one use case. It joins the two models you already have: the use case specification says what happens; the design class diagram says which object does it. In Visual Paradigm, create a UML Sequence Diagram, add the actor and lifelines, then draw messages from top to bottom and select an operation of the receiver's class for each.

Element	Notation	Meaning
Actor	Stick figure with a dashed lifeline	The user or external system that starts or serves the interaction, as in the use case diagram.
Lifeline	Box name : Class with a dashed line below	One object taking part. Time runs down the page. Internal object lifelines use classes from the class diagram; actors and external systems remain outside the system boundary.
Synchronous message	Solid line, filled arrowhead, labelled 1.2: operation(arguments)	A call. The receiver's class must have the operation.
Reply	Dashed line, open arrowhead, labelled with the value returned	Returns control and any result to the caller. Replies may be omitted when the return is clear; show important results in this lab.
Create message	Dashed line, open arrowhead, ending at the new object's box	Creates an object. Its box is drawn at the time of creation.
Self-call	Message that loops back to the same lifeline	A call to the object's own operation.

Element	Notation	Meaning
Activation	Thin bar on a lifeline	The period during which the object is carrying out a call.
Destruction	X at the end of a lifeline	The object is discarded.
alt, opt, loop	Frame with the operator in the corner and a [guard] for each operand	Alternative branches, an optional part, or a repeated part. One alt operand runs; guards must not overlap.
Numbering	1, 1.1, 1.2, 2 ...	Use message numbering or use-case step references consistently. The examples use both conventions; UML does not require numbering.

Five checks for every message

- **Source.** Follow the use-case steps. Actor requests go to the use-case controller; each system response becomes one or more messages.
- **Receiver.** Send each message to the object responsible for the action.
- **Operation.** Match the receiver's operation name and parameters. Creation calls a constructor; external calls use the agreed interface.
- **Link.** The sender needs an association, parameter, or reference to an object it created or found.
- Flow. Show alternatives with Boolean guards and step references. Make clear which branches rejoin, repeat or end the use case.

Part A QU Print Shop — Worked examples

Figures 1 and 2 show a worked design using internal controllers, stored jobsLab 6 specifications and domain objects. The table records the refinements to the Lab 7 class model.

Class refinement	Operations and access
SubmitJobController	submitJob(user, doc, options): boolean. Uses PASGateway and PrintJobRepository; creates and retains the new PrintJob.
CollectionController	recordCollection(user, job): void; issueReceipt(job): CollectionReceipt. Receives the job reference and uses PrintJobRepository.
PrintJobRepository	save(job): void records or updates a job; retains the stored PrintJob objects.
PrintJob and CollectionReceipt	Keep calculateCost() and markCollected(). Add queue(): void and attachReceipt(receipt): void to PrintJob. Constructors receive the data shown in the figures.

- Submit job: the controller creates a temporary PrintJob, asks it for its cost, and checks credit through PASGateway (UC006). Sufficient credit queues and saves the job; insufficient credit discards it and returns false.
- Collection: the controller marks the supplied printed job as collected and saves it. At "collection recorded", an optional request creates and attaches a receipt, then saves the job again.
- Why these receivers? Controllers coordinate requests; PrintJob owns its cost, status and receipt; the repository stores jobs; PASGateway communicates with PAS.

A.1 Submit a print job

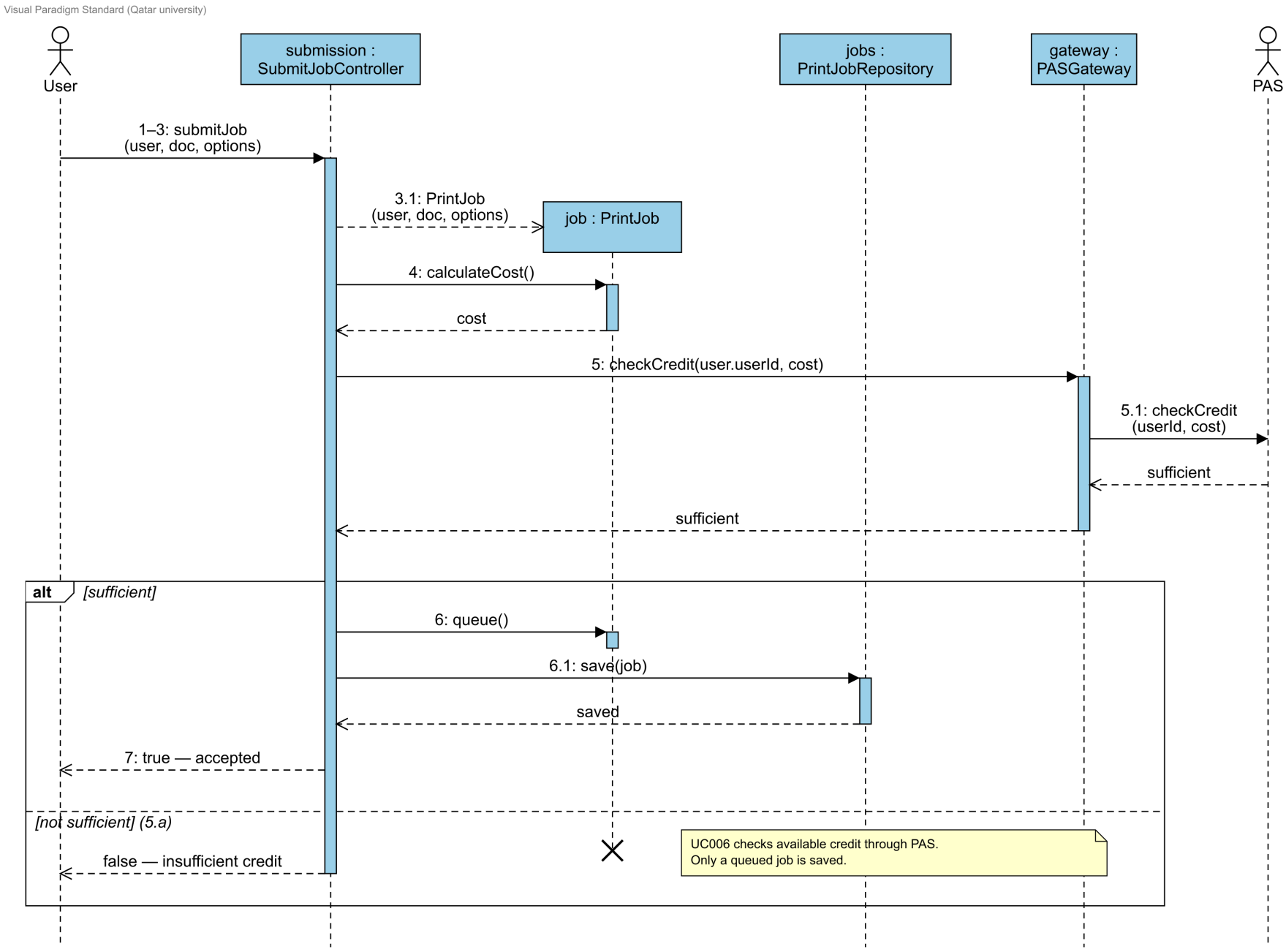


Figure 1 QU Print Shop: UC001 Submit print job

A.2 Optional receipt

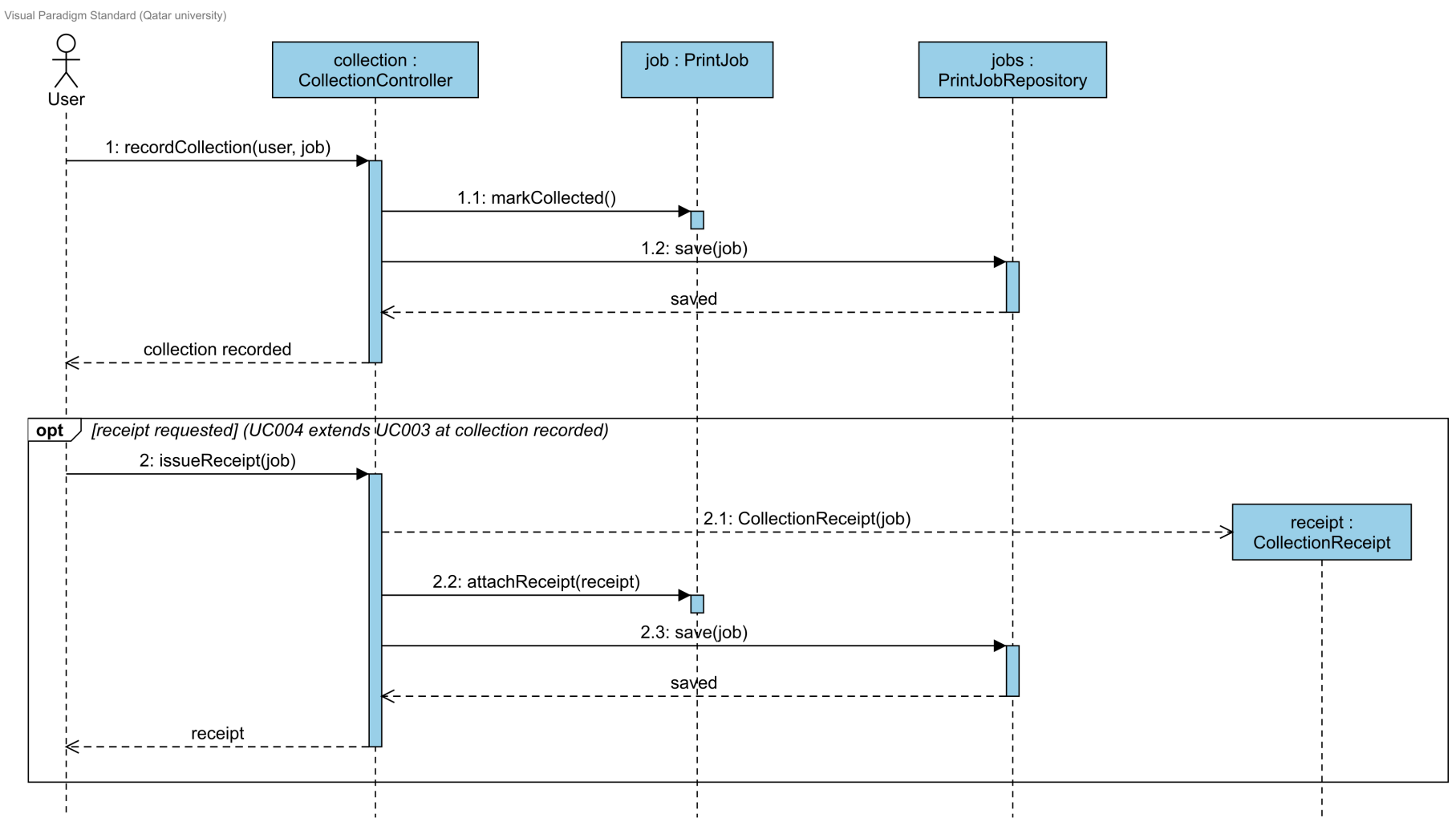


Figure 2 QU Print Shop: UC003 Collect printed document

The opt fragment implements UC004 at the “collection recorded” extension point. Collection succeeds even when no receipt is requested.

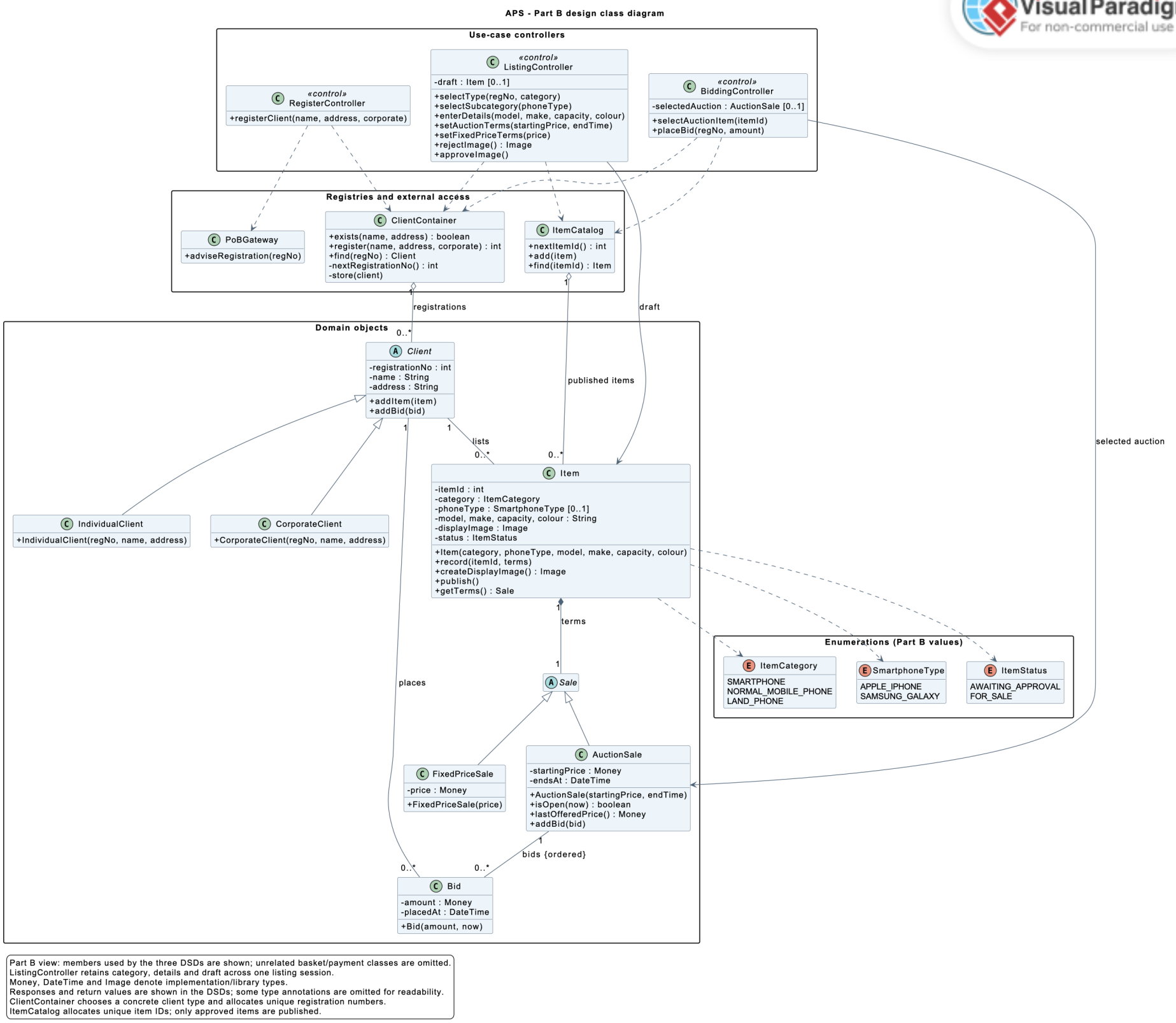
Design detail: the two controllers divide the Lab 7 PrintShop responsibilities. Submit input steps 1–3 are combined in one form request. The collection example assumes the supplied job is already printed.

Part B All Phones Shop - Individual sequence diagrams

Review the Part B class diagram (Figure 3) and the Register client worked example (Figure 4) with your instructor (10 minutes). The two task specifications are included at the end of this handout.

For UC003 List item for sale and UC006 Place bid, draw your own design sequence diagrams and prepare lab08-solution.docx using the instructions below.

Part B design class diagram



Part B view: members used by the three DSDs are shown; unrelated basket/payment classes are omitted.
ListingController retains category, details and draft across one listing session.
Money, DateTime and Image denote implementation/library types.
Responses and return values are shown in the DSDs; some type annotations are omitted for readability.
ClientContainer chooses a concrete client type and allocates unique registration numbers.
ItemCatalog allocates unique item IDs; only approved items are published.

Figure 3 APS design class diagram — Part B view

Use this class model as the starting point for Part B. ListingController retains the draft item, and BiddingController retains the selected auction between requests.

Class	Operations used by the DSDs (parameters appear in the diagrams)
RegisterController	registerClient
ListingController	selectType, selectSubcategory, enterDetails, setAuctionTerms, setFixedPriceTerms, rejectImage, approveImage
BiddingController	selectAuctionItem, placeBid
ClientContainer / ItemCatalog	exists, register, find, nextRegistrationNo, store / nextItemId, add, find
Client / Item	addItem, addBid / record, createDisplayImage, publish, getTerms
AuctionSale / PoBGateway	isOpen, lastOfferedPrice, addBid / adviseRegistration

Worked example: Register client (UC001)

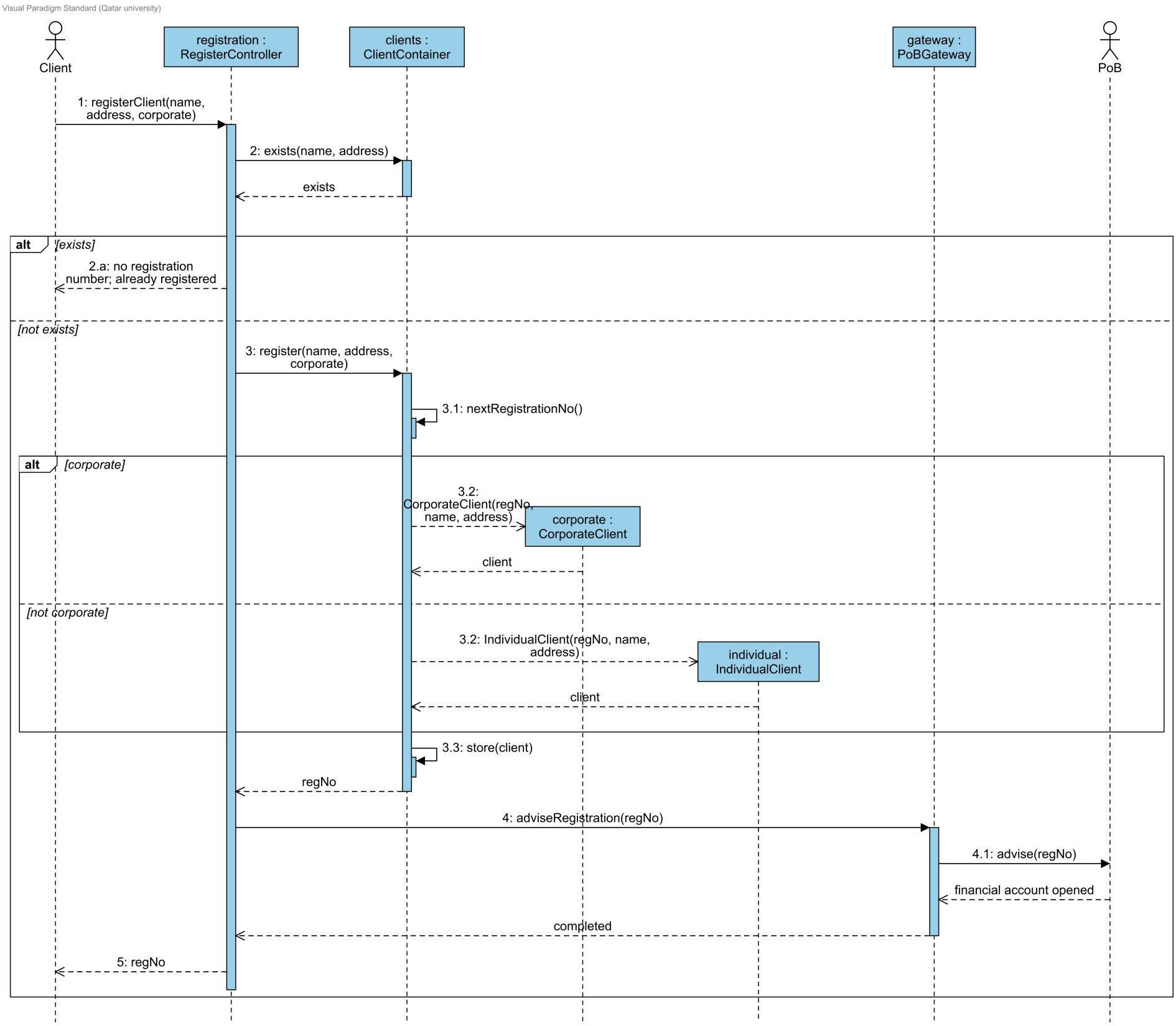


Figure 4 APS: UC001 Register client — design sequence diagram

A client provides a name and address. APS rejects an existing registration; otherwise, it assigns a unique number, notifies PoB to open the financial account, and returns the number. RegisterController coordinates these objects. The corporate flag is a design input for selecting the client subclass.

Task 1: List item for sale (UC003) - 40 minutes

- Create Table 1 for UC003 with four columns: Specification step, Message, Receiver, and Operation in the class diagram. Add one row per step and more rows where needed.
- Draw the diagram in Visual Paradigm. Show how alternative flows 1.a, 3.a and 7.a are handled, the re-creation of the display image, and the item's inclusion in the catalog.

Task 2: Place bid (UC006) - 40 minutes

- Create Table 2 for UC006 using the same four columns.
- Draw the diagram. Show the creation of the Bid, its recording in the client's registration, and alternative flows 2.a, 4.a and 4.b, including where the use case resumes or ends.

APS does not specify the first-bid rule. State your assumption; for example, compare the first bid with the starting price.

Review and update the class diagram - 15 minutes

Exchange diagrams with a classmate and apply the five checks to every message. Record one problem found and the change you made. Record required class changes and their source messages; include the updated class view in your document and .vpp, or state 'No changes required'.

Submission requirements

10 minutes. Open the CMPS310 GitHub repositories and select your own repository. Submit these two files inside a folder named Lab 8 Sequence Diagrams, using the individual repository workflow from Lab 4. Part A is not submitted. Commit and push both files, then check that they open from your repository.

File	Required contents
lab08-solution.docx	Name and student ID; Tables 1 and 2; readable images of both sequence diagrams; the review record; class diagram changes; any stated assumptions.
aps-sequence-diagrams.vpp	Editable Visual Paradigm project containing both sequence diagrams.

Assessment criteria (10 points)

Criterion	Points
Messages trace the specification: every actor action and system response is covered, in order, with step references	3
Messages are consistent with the class diagram: correct receivers, operations, arguments and links; the class diagram is updated where needed	3
Alternative flows, object creation and replies are modelled correctly using suitable fragments and guards, with clear continuation or termination	2.5
Correct notation and numbering, readable diagrams, review record and complete submission	1.5

Further reading: Visual Paradigm: What is Sequence Diagram?; OMG UML 2.5.1, Clause 17 Interactions

UC003 List item for sale specification

Use case ID:	**UC003 List item for sale**
Brief description	A registered client lists a phone for sale by auction or at a fixed price. APS records the item, prepares a display image for the seller's approval and includes the approved item in the product catalog.
Primary actors	Client, acting as seller.
Secondary (supporting) actors	None.
Trigger	The client chooses to sell an item.
Preconditions	Login has been checked outside APS. The client has a registration.
Postconditions	The item is recorded with its ID, details, sale type and terms, attached to the client's registration, and displayed in the product catalog with the approved image.
Main Success Scenario	
Actor Action	**System Response**
1. The client selects the type of item: smartphone, normal mobile phone or land phone (see 1.a).	
2. The client provides the model, make, capacity and colour.	
3. The client selects sale by auction and gives the starting price and the auction end time (see 3.a).	4. APS assigns an ID to the item. 5. APS records the item information and sale terms and attaches them to the client's registration. 6. APS creates a display image from the item description and asks the client to approve it.
7. The client approves the display image (see 7.a).	8. APS includes the item in the product catalog and displays it for sale.
Alternative flows: 1.a. Smartphone: the client also selects the sub-category, Apple iPhone or Samsung Galaxy. The use case resumes at step 2. 3.a. Fixed-price sale: the client selects fixed-price sale and gives the price. The use case resumes at step 4. 7.a. Image not approved: APS re-creates the display image and asks the client again. The use case resumes at step 7.	

UC006 Place bid specification

Use case ID:	**UC006 Place bid**
Brief description	A registered client offers a price for an item on auction. APS accepts the bid if the auction is still open and the bid is greater than the last offered bid, and records it in the client's registration.
Primary actors	Client, acting as buyer.
Secondary (supporting) actors	None.
Trigger	The client selects an item on auction to bid on.
Preconditions	Login has been checked outside APS. The client has a registration. The item is displayed in the catalog for auction.
Postconditions	The bid is recorded in the client's registration and is the last offered bid for the item.
Main Success Scenario	
Actor Action	**System Response**
1. The client selects an item on auction.	2. APS checks that the auction end time has not passed and shows the last offered bid (see 2.a).
3. The client submits a bid price.	4. APS checks that the auction end time has not passed and that the bid is greater than the last offered bid (see 4.a and 4.b). 5. APS records the bid in the client's registration as the last offered bid. 6. APS confirms the accepted bid to the client.
Alternative flows: 2.a. Auction ended: APS informs the client that the auction has ended. No bid is recorded and the use case ends. 4.a. Auction ended before the bid was submitted: as 2.a. 4.b. Bid not greater than the last offered bid: APS rejects the bid and informs the client. The use case resumes at step 3.	